PERTEMUAN 11
REGRESI

I. LANDASAN TEORI

1.1. Konsep Dasar Analisis Regresi

Regresi merupakan salah satu teknik *supervised learning* dalam pembelajaran mesin (*machine learning*) yang digunakan untuk memprediksi nilai kontinu (*continuous numerical values*) berdasarkan satu atau beberapa variabel independen (fitur). Berbeda dengan klasifikasi yang mengelompokkan data ke dalam kategori, regresi fokus pada pencarian pola hubungan numerik antara fitur X dan target y. Contoh penerapan regresi antara lain prediksi harga saham, prediksi harga emas, perkiraan suhu, hingga estimasi harga properti.

1.2. Linear Regression (Regresi Linear)

Linear Regression adalah algoritma regresi paling dasar yang memodelkan hubungan antara variabel independen (X) dan variabel dependen (y) secara linear.
Persamaan umum regresi linear sederhana dirumuskan sebagai:

$$y = \beta_0 + \beta_1 X + \epsilon \qquad 11.1$$

Di mana:

y = Nilai variabel target/dependen.
X = Nilai variabel fitur/independen.
β_0 = *Intercept* (titik potong sumbu y saat $X = 0$).
β_1 = Koefisien regresi (kemiringan/slop garis).
ϵ = *Error* atau residu.

Tahapan Alur Kerja Regresi

Berdasarkan modul praktikum, alur standar pemodelan regresi terdiri dari:

1. **Persiapan Data:** Pengambilan dataset (*time-series* atau data sekunder) dan mendefinisikan variabel X (fitur) serta y (target).
2. **Split Data:** Membagi dataset menjadi *Training Set* (misal 80%) untuk melatih model dan *Testing Set* (misal 20%) untuk pengujian.
3. **Pelatihan Model (*Training*):** Menyesuaikan garis regresi ke data latih menggunakan fungsi .fit().
4. **Prediksi dan Evaluasi (*Testing*):** Menguji performa model terhadap data uji dan menghitung metrik evaluasi kesalahan/akurasi.
5. **Visualisasi:** Menampilkan perbandingan antara data aktual (*scatter plot*) dan garis prediksi model.

1.3. Algoritma Ensemble Learning (Advanced Regression)

Ensemble learning menggabungkan beberapa model *weak learners* (biasanya berupa *Decision Tree*) untuk membentuk satu model regresi yang lebih kuat dan presisi.

1.3.1. Gradient Boosting Regressor

Gradient Boosting membangun model secara berurutan (*sequential*). Setiap pohon (*tree*) baru dilatih untuk memperbaiki kesalahan (*residu*) yang dihasilkan oleh pohon sebelumnya

menggunakan teknik *gradient descent* pada fungsi kerugian (*loss function*).

1.3.2. XGBoost (Extreme Gradient Boosting)

XGBoost adalah implementasi teroptimasi dari *Gradient Boosting*. Algorithm ini menawarkan performa tinggi melalui:

1. **Regulerisasi (L1/L2):** Mencegah terjadinya *overfitting*.
2. **Paralelisasi:** Proses komputasi yang efisien dan cepat.
3. **Handling Missing Values:** Penanganan otomatis terhadap data yang hilang.

1.3.3. LightGBM (Light Gradient Boosting Machine)

LightGBM dikembangkan oleh Microsoft dengan fokus pada kecepatan dan efisiensi memori. Perbedaannya dengan algoritma *boosting* lain adalah:

- **Leaf-wise Tree Growth:** Mengembangkan pohon berdasarkan *leaf* yang menghasilkan reduksi *loss* terbesar (bukan *level-wise*), sehingga lebih presisi pada dataset besar.
- **Histogram-based Algorithm:** Mempercepat pengelompokan nilai fitur kontinu ke dalam bin terdiskritisasi.

1.3.4. AdaBoost (Adaptive Boosting Regressor)

AdaBoost bekerja dengan memberikan bobot lebih besar pada sampel data yang memiliki kesalahan prediksi (*error*) tinggi dari pembelajar sebelumnya. Pohon-pohon selanjutnya difokuskan untuk memperbaiki data yang sulit diprediksi tersebut.

1.4. Metrik Evaluasi Regresi

Untuk mengukur seberapa presisi model regresi dalam memprediksi data, digunakan metrik evaluasi berikut:

1. **Mean Absolute Error (MAE):** Rata-rata dari nilai absolut selisih antara nilai prediksi dan nilai aktual.
$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i| \quad (11.2)$$
2. **Mean Squared Error (MSE):** Rata-rata dari kuadrat selisih antara nilai prediksi dan nilai aktual. Memberikan penalti lebih besar pada nilai margin *error* yang ekstrem.
$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \quad (11.3)$$
3. **Root Mean Squared Error (RMSE):** Akar kuadrat dari MSE. Mengembalikan nilai metrik ke dalam skala unit yang sama dengan variabel target y.
$$\text{RMSE} = \sqrt{\text{MSE}} \quad (11.4)$$
4. **Mean Absolute Percentage Error (MAPE):** Rata-rata persentase kesalahan antara nilai prediksi dan aktual. Metrik ini memudahkan interpretasi kesalahan dalam bentuk persentase ($0\% - 100\%$).
$$\text{MAPE} = \frac{100\%}{n}\sum_{i=1}^{n} \left|\frac{y_i - \hat{y}_i}{y_i}\right| \quad (11.5)$$
5. **R-Squared Score (R^2 Score):** Mengukur seberapa baik persentase varians dari variabel dependen dapat dijelaskan oleh variabel independen. Rentang nilai R^2 berkisar antara 0 hingga 1 (atau $0\% - 100\%$). Semakin mendekati 1, semakin baik performa model.
$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2} \quad (11.6)$$

1.5. Perancangan UI/UX untuk Model Machine Learning

Pengembangan sistem pembelajaran mesin yang utuh tidak hanya berhenti pada pelatihan model, tetapi juga mencakup antarmuka pengembang/pengguna (*User Interface/User Experience*).

- **User Interface (UI):** Tampilan visual yang dirancang (misalnya menggunakan *Streamlit*, *Gradio*, atau *Tkinter*) agar pengguna dapat memasukkan variabel input secara intuitif.
- **User Experience (UX):** Pengalaman pengguna saat berinteraksi dengan aplikasi, memastikan proses prediksi berjalan lancar, cepat, mudah dipahami, serta menyajikan hasil prediksi dan visualisasi evaluasi secara informatif.

Berikut adalah tabel perbandingan antar metode regresi berbasis *Ensemble Learning* yang dapat Anda masukkan ke dalam bagian pembahasan landasan teori:

Perbandingan Metode Ensemble Learning untuk Regresi

Fitur / Karakteristik	XGBoost (Extreme Gradient Boosting) TM11 Regresi.pdf	LightGBM (Light Gradient Boosting Machine) TM11 Regresi.pdf	Gradient Boosting Regressor TM11 Regresi.pdf	AdaBoost (Adaptive Boosting) TM11 Regresi.pdf
Pendekatan Utama	*Gradient Boosting* teroptimasi dengan *regularization*	*Gradient Boosting* berbasis *histogram*	*Gradient Boosting* standar (berurutan)	*Adaptive Boosting* berbasis pembuat bobot (*re-weighting*)
Pertumbuhan Pohon (*Tree Growth*)	*Level-wise* / *Depth-wise* (melebar secara vertikal per level)	*Leaf-wise* (memilih daun dengan penurunan *loss* terbesar)	*Level-wise* (melebar secara seimbang)	*Level-wise* (biasanya pohon dangkal / *stumps*)
Kecepatan & Efisiensi	Sangat cepat & efisien dalam memori	Sangat cepat & paling hemat memori pada data besar	Lebih lambat pada data berukuran besar	Cepat untuk dataset kecil hingga sedang
Pencegahan Overfitting	Memiliki regularisasi L1 (Lasso) & L2 (Ridge) bawaan	Menggunakan pembatasan *max depth* / *num leaves*	Memerlukan *hyperparameter tuning* manual	Rentan terhadap *noise* & data pencilan (*outliers*)
Penanganan Missing Value	Penanganan otomatis secara internal	Penanganan otomatis secara internal	Harus diimputasi/dibersihkan terlebih dahulu	Harus diimputasi/dibersihkan terlebih dahulu
Skala Data Ideal	Data tabular sedang hingga besar	Data sangat besar (*large-scale data*)	Data skala kecil hingga sedang	Data skala kecil hingga sedang

Kesimpulan

- XGBoost vs LightGBM: Kedua algoritma ini berfokus pada efisiensi tinggi dan performa prediksi maksimal. XGBoost sangat unggul dalam mencegah *overfitting* berkat fitur regularisasi bawaannya, sedangkan LightGBM sangat unggul dalam kecepatan pemrosesan data berjumlah besar berkat teknik *leaf-wise tree growth*.
- AdaBoost vs Gradient Boosting: Gradient Boosting bekerja secara bertahap meminimalkan fungsi kerugian (*loss function*) menggunakan *gradient descent*, sedangkan AdaBoost berfokus pada pembobotan ulang sampel data yang salah diprediksi pada iterasi sebelumnya.

II. PRAKTIKUM

Berikut script Python untuk Regresi

```
# Prediksi Harga Saham / Emas dengan Linear Regression
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split from
sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
# ----------------------------------------------------
# 1 Contoh: Membuat dataset dummy (bisa diganti CSV nyata) # -------
----------------------------------------------
# Misal: kolom 'Date' dan 'Close' (harga penutupan)
np.random.seed(42)
days = np.arange(1, 101) # 100 hari
price = 100 + 0.5 * days + np.random.normal(0, 3, size=len(days)) # harga naik Perlahan data =
pd.DataFrame({'Day': days, 'Close': price})

# Jika punya file CSV asli, bisa ganti dengan:
# data = pd.read_csv('harga_emas.csv') print(data.head())
# ----------------------------------------------------
# 2 Persiapan fitur dan target
# ----------------------------------------------------
X = data[['Day']]        # fitur (misal: waktu)
y = data['Close']        # target (harga penutupan)

# ----------------------------------------------------
# 3 Split data (80% training, 20% testing) # ---
----------------------------------------------
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# ----------------------------------------------------
# 4 Buat dan latih model Linear Regression # ----
----------------------------------------------
model = LinearRegression() model.fit(X_train,
y_train)

# ----------------------------------------------------
# 5 Prediksi dan evaluasi
# ----------------------------------------------------
y_pred = model.predict(X_test)

# Evaluasi performa model
mae = mean_absolute_error(y_test, y_pred) mse
= mean_squared_error(y_test, y_pred) rmse =
np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\n=== Hasil Evaluasi Model ===") print(f"Mean
```

```
Absolute Error (MAE): {mae:.3f}") print(f"Mean Squared
Error (MSE): {mse:.3f}") print(f"Root MSE (RMSE):
{rmse:.3f}")
print(f"R² Score: {r2:.3f}")

# ----------------------------------------------------
# 6 Visualisasi hasil prediksi
# ----------------------------------------------------
plt.figure(figsize=(10, 6))
plt.scatter(X_test, y_test, color='blue', label='Data Aktual')
plt.plot(X_test, y_pred, color='red', linewidth=2, label='Prediksi Linear Regression') plt.title("Prediksi
Harga (Linear Regression)")
plt.xlabel("Hari")
plt.ylabel("Harga")
plt.legend()
plt.grid(True)
plt.show()

# ----------------------------------------------------
# 7 Contoh prediksi hari ke-110
# ----------------------------------------------------
day_baru = np.array([[110]]) prediksi_harga =
model.predict(day_baru)
print(f"\nPrediksi harga untuk hari ke-110: {prediksi_harga[0]:.2f}")
```

Tugas :

1. Buatlah program untuk prediksi harga menggunakan XGBoost, LightGBM → Kelompok Ganjil
2. Buatlah program untuk prediksi harga menggunakan AdaBoost (Adaptive Boosting), Gradient Boosting → Kelompok Genap
3. Data bebas tapi dalam satu kelas datanya harus sama
4. Gunakan data sekunder yang diperoleh dari online
5. Evaluasi yang digunakan MAE, MAPE, RMSE, MSE, dan R2
6. Buatlah UI/UX dari model yang memiliki performa yang terbaik
7. Buatlah laporan dari program diatas berisi screen shot dan penjelasan dari Proses training Proses testing (evaluasi), dan UI/UX
8. Beri nama laporan tersebut dengan nama **Tugas TM11_Kelompok #.pdf**
9. File yang dikumpulkan Adalah laporan, dan koding
10. Dikumpulkan pada hari **Minggu, 8 November 2026 pukul 23.59 WIB**